

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

|                |                                                                                                                  |               |                                                    |
|----------------|------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------------|
| Course Code:   | CSA10 / CSA1063                                                                                                  | Course Title: | Software Engineering                               |
| CO Assessed:   | CO4                                                                                                              | Assessment:   | Assessment Tool 2 – CI/CD Pipeline Design Exercise |
| Weightage:     | 20%                                                                                                              | Total Marks:  | 25                                                 |
| CO4 Statement: | Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics |               |                                                    |
| Student Name:  |                                                                                                                  | Reg. No.:     |                                                    |
| Date:          |                                                                                                                  | Duration:     | 60 minutes                                         |

### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

## Annexure A – CI/CD Pipeline Design Exercise

### Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

**Deliverable:** Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

# 1. ANALYZE THE PROJECT REQUIREMENTS AND IDENTIFY THE CI/CD NEEDS OF THE ASSIGNED PROBLEM STATEMENT

## 1.1 Introduction to the Project

The **AI-Based Personal Finance and Budget Planning System** is designed to help users manage their personal financial activities through an intelligent software platform. The system allows users to record income and expenses, categorize transactions, create budgets, monitor spending, analyze financial behavior, and receive AI-based recommendations.

The main purpose of the system is to provide users with a centralized platform for understanding and managing their financial activities. Instead of manually calculating expenses and savings, the proposed system can automatically process financial information and generate useful insights.

## 1.2 Problem Statement

The proposed system addresses these problems by providing:

- Automated income and expense tracking
- Budget creation
- Expense categorization
- Spending analysis
- Financial dashboards
- AI-based recommendations
- Savings analysis
- Financial reports
- Automated alerts

For example, modifying the expense calculation module could affect:

- Monthly budget calculation
- Savings calculation
- Dashboard values
- AI recommendations
- Financial reports

## 1.3 Functional Requirements

The major functional requirements are:

### User Management

The system should allow users to:

- Register an account
- Login securely
- Logout
- Manage their profile

- Maintain authentication sessions

### Income Management

Users should be able to:

- Add income
- Edit income
- Delete income
- View income history
- Categorize income

### Expense Management

Users should be able to:

- Add expenses
- Edit expenses
- Delete expenses
- View transaction history
- Categorize expenses

### Budget Management

- Create monthly budgets
- Set category-wise limits

### AI-Based Analysis

- Identify spending patterns
- Predict future expenses
- Recommend budget adjustments

## 1.4 Non-Functional Requirements

The system must also satisfy non-functional requirements.

### Security

Financial information must be protected from unauthorized access.

### Performance

The dashboard and financial calculations should respond within an acceptable time.

### Reliability

Financial calculations must produce consistent and accurate results.

## Scalability

The system should support an increasing number of users and transactions.

### 1.5 CI/CD Requirements

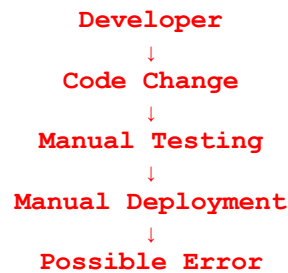
Based on these project requirements, the following CI/CD capabilities are required:

1. Source-code version control
2. Automated build
3. Automated testing
4. Code-quality analysis
5. Security scanning

### 1.6 Why CI/CD is Required

CI/CD is important because the project may be developed and updated continuously.

Without CI/CD:



#### With CI/CD:



The CI/CD approach therefore reduces manual errors and improves the reliability of the finance application.

## 2. DESIGN THE CI/CD PIPELINE WORKFLOW SHOWING STAGES SUCH AS SOURCE CODE MANAGEMENT, BUILD, TESTING, CODE QUALITY ANALYSIS, PACKAGING, AND DEPLOYMENT

### 2.1 Overview of the Pipeline

The CI/CD pipeline provides an automated workflow from source-code modification to application deployment.

The proposed pipeline contains the following major stages:

1. Source Code Management
2. Code Checkout
3. Dependency Installation
4. Build
5. Unit Testing
6. Integration Testing
7. Code Quality Analysis
8. Security Scanning
9. Packaging
10. Artifact Storage
11. Staging Deployment
12. Acceptance Testing
13. Production Deployment
14. Monitoring
15. Rollback

### 2.2 Source Code Management

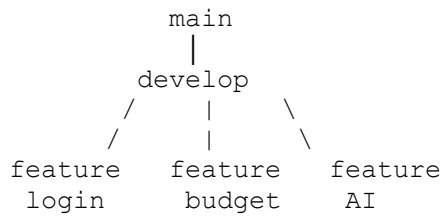
Git is used for source-code version control and GitHub can be used as the central repository.

The project repository can be organized as:

```
AI-Personal-Finance/  
├── frontend/  
├── backend/  
├── ai-model/  
├── database/  
├── tests/  
├── requirements.txt  
├── Dockerfile  
├── docker-compose.yml  
├── README.md  
├── .github/  
│   └── workflows/  
│       └── ci-cd.yml
```

### 2.3 Branching Workflow

A suitable branching strategy is:



Feature branches are used for individual functionalities.

Examples:

```
feature/login
feature/expense
feature/budget
feature/ai-analysis
```

After implementation, the developer creates a Pull Request.

The CI pipeline automatically validates the changes.

## 2.4 Build Stage

After source-code checkout, the pipeline installs dependencies and builds the application.

For a Python-based application:

```
Checkout Code
  ↓
Setup Python
  ↓
Install Dependencies
  ↓
Validate Code
  ↓
Build Application
```

The build stage checks:

- Python version
- Required libraries
- Source-code syntax
- Configuration files
- Dependency availability

## 2.5 Testing Stage

The testing stage contains:

- Unit testing
- Integration testing
- API testing

- Database testing
- Regression testing

Financial calculations should be tested carefully.

Example:

Income = ₹50,000  
Expenses = ₹30,000

Expected Savings = ₹20,000

## 2.6 Code Quality Analysis

After testing, the source code is analyzed for:

- Bugs
- Code smells
- Duplicate code
- Maintainability problems
- Security issues

SonarQube can be integrated into the pipeline.

## 2.7 Packaging

After successful quality checks, the application can be packaged using Docker.

```
Source Code
  ↓
Docker Build
  ↓
Docker Image
  ↓
Container Registry
```

## 2.8 Deployment

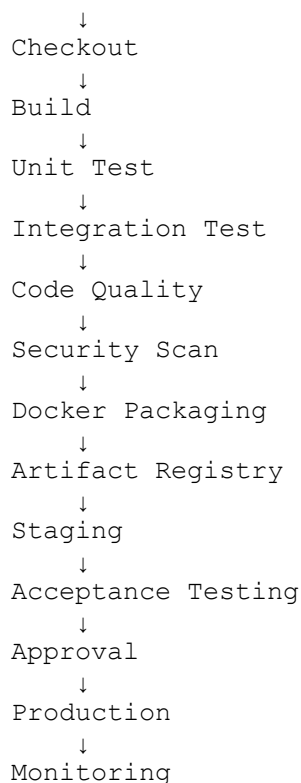
The application is first deployed to the staging environment.

After successful staging tests:

```
Staging
  ↓
Approval
  ↓
Production
```

## 2.9 Complete Workflow

```
Developer
  ↓
Git / GitHub
  ↓
Commit / Pull Request
  ↓
CI Trigger
```



This workflow ensures that only validated software reaches production.

---

### 3. IDENTIFY SUITABLE TOOLS AND TECHNOLOGIES FOR EACH STAGE OF THE PIPELINE AND JUSTIFY THEIR SELECTION

#### 3.1 Tool Selection

The following tools are suitable for the proposed CI/CD pipeline.

| Stage                 | Tool                     | Purpose                        |
|-----------------------|--------------------------|--------------------------------|
| <b>Source Control</b> | Git                      | Version management             |
| <b>Repository</b>     | GitHub                   | Code hosting and collaboration |
| <b>CI/CD</b>          | GitHub Actions           | Pipeline automation            |
| <b>Programming</b>    | Python                   | Application development        |
| <b>Testing</b>        | pytest                   | Automated testing              |
| <b>Code Quality</b>   | SonarQube                | Static analysis                |
| <b>Linting</b>        | Ruff / Flake8            | Coding standards               |
| <b>Security</b>       | SAST/Dependency Scanner  | Vulnerability detection        |
| <b>Packaging</b>      | Docker                   | Containerization               |
| <b>Registry</b>       | Container Registry       | Image storage                  |
| <b>Deployment</b>     | Cloud/Container Platform | Application hosting            |
| <b>Monitoring</b>     | Monitoring Platform      | System health                  |
| <b>Notifications</b>  | Email/Team Tool          | Alerts                         |

#### 3.2 Git



Git is selected because it provides:

- Version control
- Branching
- Merging
- Commit history
- Revert capability
- Collaboration

If a new change causes an issue, previous versions can be identified and restored.

### 3.3 GitHub

GitHub is suitable for storing the project repository.

It supports:

- Pull Requests
- Branches
- Issues
- Code reviews
- Repository management
- CI/CD integration

### 3.4 GitHub Actions

GitHub Actions is suitable for automating the CI/CD workflow.

- Code is pushed
- Pull Request is created
- A release is created

### 3.5 pytest

pytest is suitable for testing Python applications.

It can be used for:

- Unit testing
- Integration testing
- Regression testing
- API testing

### 3.6 SonarQube

SonarQube is useful for static code analysis.

It identifies:

- Bugs
- Vulnerabilities
- Code smells
- Duplicated code
- Maintainability problems

### 3.7 Docker

Development  
↓  
Testing  
↓  
Staging  
↓  
Production

### 3.8 Container Registry

personal-finance:v1.0  
personal-finance:v1.1  
personal-finance:v1.2

This makes version management and rollback easier.

### 3.9 Cloud Platform

A cloud or container platform can host the staging and production application.

The selected platform should support:

- Container deployment
- Environment variables
- Scaling

### 3.10 Monitoring Tools

Monitoring tools can track:

- CPU usage
- Memory
- Response time
- Error rate
- 

### 3.11 Tool Selection Summary

The selected tools are appropriate because they provide an integrated workflow:

GitHub  
↓  
GitHub Actions  
↓  
pytest

↓  
SonarQube  
↓  
Security Scanner  
↓  
Docker  
↓  
Container Registry  
↓  
Cloud Deployment

---

## 4. DEFINE APPROPRIATE AUTOMATED TESTING STRATEGIES TO BE INTEGRATED INTO THE PIPELINE

### 4.1 Importance of Automated Testing

Automated testing is essential for the AI-based personal finance system because incorrect calculations may provide users with inaccurate financial information.

Every major software modification should trigger automated tests.

### 4.2 Unit Testing

Unit testing checks individual functions.

Examples:

- Income calculation
- Expense calculation
- Budget calculation
- Savings calculation
- Transaction categorization

Example:

Input:  
Income = ₹60,000  
Expense = ₹40,000

Expected:  
Savings = ₹20,000

The test verifies whether the implemented function returns the expected result.

### 4.3 Integration Testing

Integration testing checks whether different modules work together.

Examples:

Expense Module  
↓

Budget Module



Dashboard

Another example:

Transaction Data



AI Model



Recommendation

## 4.4 API Testing

Backend APIs should be automatically tested.

Examples:

```
POST /login
POST /expense
GET /expense
POST /budget
GET /dashboard
```

Tests should verify:

- Status code
- Response format
- Response data
- Authentication
- Error handling

## 4.5 Database Testing

Database testing verifies:

- Insert operations
- Update operations
- Delete operations
- Retrieval
- Constraints
- Data consistency

Example:

When a user adds an expense, the transaction should be correctly stored in the database.

## 4.6 Regression Testing

Regression testing ensures that an existing feature is not broken by a new change.

Example:

If the developer modifies the expense module, the pipeline should also verify:

- Budget
- Dashboard
- Savings
- Reports

## 4.7 Boundary Testing

Financial applications should test boundary values.

## 4.8 Security Testing

The pipeline should test for:

- Weak authentication
- Unauthorized access
- Invalid input
- SQL injection
- Hard-coded secrets
- Vulnerable dependencies

## 4.9 Smoke Testing

After staging deployment, smoke tests verify whether the basic application is working.

For example:

```
Application Available?  
  ↓  
Login Working?  
  ↓  
Dashboard Working?  
  ↓  
Database Connected?
```

## 4.10 Acceptance Testing

Acceptance testing verifies whether the application satisfies the expected requirements.

For example:

- User can add income.
- User can add expenses.
- Budget can be created.
- Dashboard displays financial information.
- AI recommendation is generated.

## 4.11 Testing Pipeline

Build

↓

Unit Test  
↓  
Integration Test  
↓  
API Test  
↓  
Database Test  
↓  
Regression Test  
↓  
Security Test  
↓  
Smoke Test  
↓  
Acceptance Test

If critical tests fail, the pipeline must stop.

---

## 5. DESIGN THE DEPLOYMENT STRATEGY, INCLUDING DEVELOPMENT, TESTING/STAGING, AND PRODUCTION ENVIRONMENTS

### 5.1 Deployment Environment Architecture

The proposed system uses three major environments:

1. Development
2. Testing/Staging
3. Production

The separation reduces the risk of deploying unfinished software directly to users.

### 5.2 Development Environment

The development environment is used by developers to implement features.

Example:

Developer  
↓  
Feature Branch  
↓  
Development Environment

Developers can test new functionality without affecting real users.

### 5.3 Testing Environment

The testing environment validates integrated application functionality.

Activities include:

- Automated tests

- Integration testing
- API testing
- Database testing
- Regression testing

## 5.4 Staging Environment

The staging environment should be similar to production.

It is used for:

- Final testing
- Smoke testing
- Acceptance testing
- Performance checks
- Deployment verification

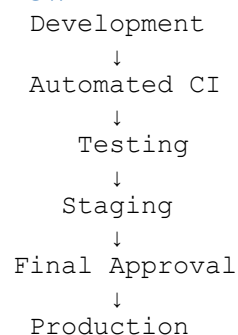
## 5.5 Production Environment

Production is the live environment used by users.

The production environment must have:

- Strong authentication
- HTTPS
- Database backup
- Monitoring
- Logging
- Secure configuration
- Rollback support

## 5.6 Environment Flow



## 5.7 Deployment Strategy

A controlled deployment strategy should be used.

### Rolling Deployment

New application instances are gradually deployed.

Advantages:

- Reduced downtime
- Controlled release
- Easy monitoring

## Blue-Green Deployment

Two environments can be maintained:

Blue → Current Production  
Green → New Version

After validation, traffic is moved to the new environment.

## Canary Deployment

A small percentage of traffic is first sent to the new version.

95% → Existing Version  
5% → New Version

If the new version performs correctly, traffic can gradually increase.

## 5.8 Recommended Deployment

For this financial application, a **controlled rolling or blue-green deployment** is appropriate because it provides reduced downtime and an easier rollback mechanism.

## 5.9 Database Deployment

Database changes should be carefully managed.

The deployment process should include:

1. Database backup
2. Migration validation
3. Migration execution
4. Application deployment
5. Database verification

Financial transaction data should never be accidentally deleted during deployment.

---

## 6. INCORPORATE APPROPRIATE SECURITY, QUALITY CHECKS, NOTIFICATIONS, AND ROLLBACK MECHANISMS IN THE PIPELINE

### 6.1 Security Requirements

Security is a critical requirement because the application handles financial and personal information.



Security checks should be included at multiple stages.

## 6.2 Secret Management

Sensitive information should not be stored directly in source code.

Examples:

```
Database Password
API Key
JWT Secret
Cloud Credentials
Encryption Key
```

These should be stored securely as environment variables or repository secrets.

## 6.3 Dependency Scanning

Third-party dependencies should be scanned for known vulnerabilities.

The pipeline should stop if a critical vulnerability is detected.

## 6.4 Static Security Analysis

Static analysis can identify:

- SQL injection
- Hard-coded passwords
- Unsafe input
- Authentication weaknesses
- Insecure coding patterns

## 6.5 Container Security

Docker images should be scanned before deployment.

The pipeline should verify:

- Base-image vulnerabilities
- Package vulnerabilities
- Security configuration

## 6.6 Quality Gates

A quality gate can define minimum release conditions.

Example:

```
Build                → PASS
Unit Tests           → PASS
```

```
Integration Tests → PASS
Code Quality      → PASS
Security          → PASS
Critical Bugs     → 0
```

If any critical condition fails:

```
Pipeline
↓
Quality Gate
↓
FAIL
↓
STOP
```

## 6.7 Notifications

The CI/CD system should notify the development team about important events.

Notifications should be generated for:

- Build success
- Build failure
- Test failure
- Security failure
- Quality-gate failure
- Staging deployment
- Production deployment
- Rollback

Example:

```
Developer
↓
Commit
↓
Pipeline
↓
Test Failure
↓
Notification
↓
Developer Fixes Code
```

## 6.8 Rollback Mechanism

Rollback restores the previous stable version when a deployment fails.

Example:

```
Current Version = v1.2.0
Previous Version = v1.1.0
```

If v1.2.0 fails:

v1.2.0  
↓  
Failure  
↓  
Rollback  
↓  
v1.1.0

## 6.9 Rollback Conditions

Rollback may occur when:

- Application crashes
- Critical API fails
- Database errors occur
- Error rate increases

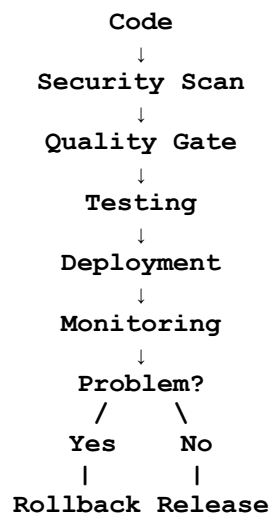
## 6.10 Monitoring After Deployment

The application should be monitored continuously.

Important metrics include:

- CPU usage
- Memory usage
- Response time

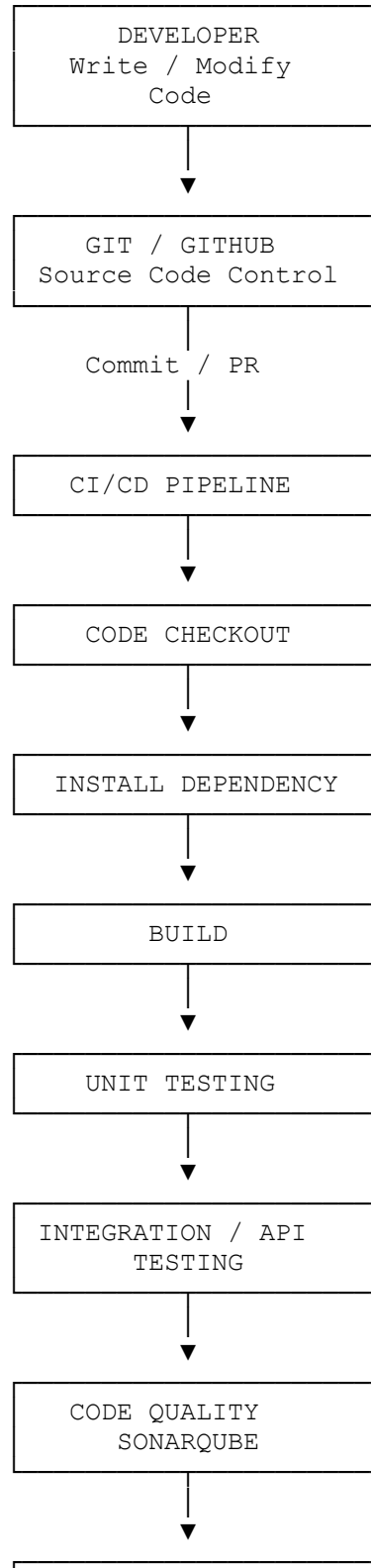
## 6.11 Complete Security and Recovery Flow

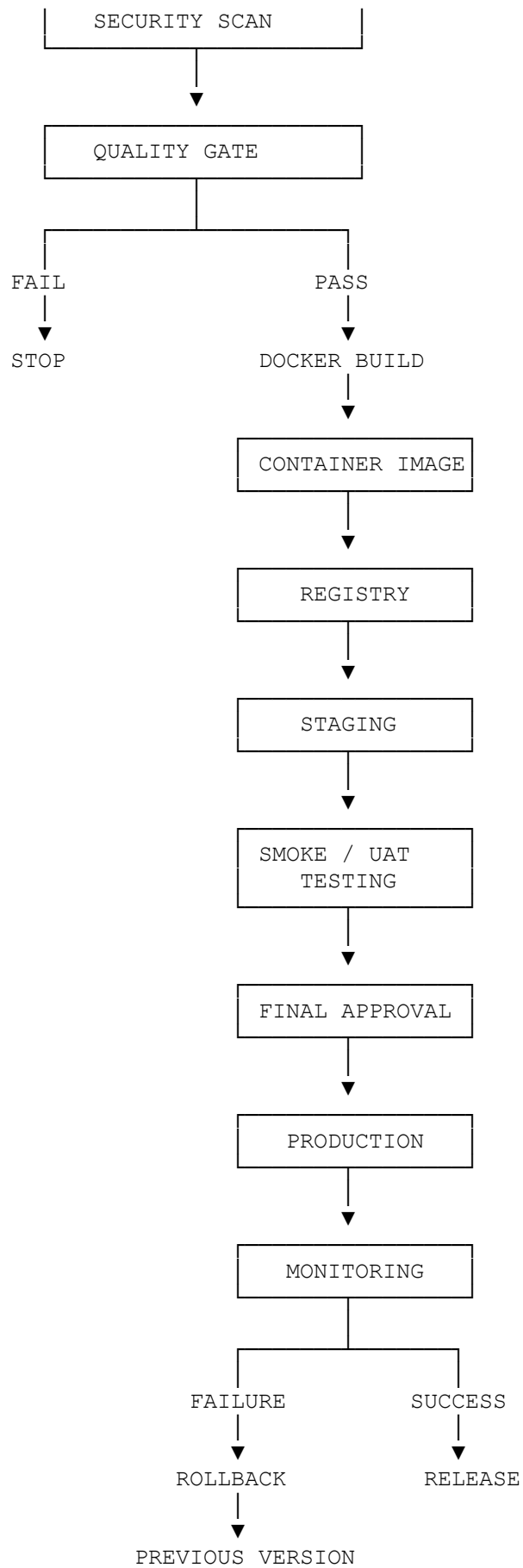


This approach improves application security, quality, reliability, and availability.

## 7. PREPARE A CI/CD PIPELINE DIAGRAM AND EXPLAIN THE WORKFLOW FROM CODE COMMIT TO DEPLOYMENT

### 7.1 Complete CI/CD Pipeline Diagram





## 7.2 Step-by-Step Workflow Explanation

### Step 1 – Code Development

The developer develops or modifies a feature of the AI-based personal finance system.

Examples:

- Login feature
- Expense management
- Budget calculation
- AI recommendation

### Step 2 – Commit

The developer commits the changes to Git.

```
git add .  
git commit  
git push
```

The changes are pushed to GitHub.

### Step 3 – Pipeline Trigger

The GitHub Actions workflow is automatically triggered.

### Step 4 – Checkout

The pipeline retrieves the latest source code.

### Step 5 – Dependency Installation

Required project libraries are installed.

### Step 6 – Build

The application is built and validated.

### Step 7 – Automated Testing

Unit, integration, API, database, and regression tests are executed.

### Step 8 – Code Quality Analysis

- Bugs
- Code smells
- Security problems
- Duplicate code

## Step 9 – Security Scan

Dependencies, source code, secrets, and containers are checked for vulnerabilities.

## Step 10 – Quality Gate

|          |        |
|----------|--------|
| Tests    | → PASS |
| Quality  | → PASS |
| Security | → PASS |
| Build    | → PASS |

Only when all critical checks pass does the pipeline continue.

## Step 11 – Packaging

The application is packaged as a Docker image.

## Step 12 – Artifact Storage

The Docker image is stored in a container registry.

## Step 13 – Staging Deployment

The exact approved application package is deployed to the staging environment.

## Step 14 – Acceptance Testing

Final smoke and acceptance tests are executed.

## Step 15 – Production Approval

After successful staging validation, production deployment is approved.

## Step 16 – Production Deployment

The application is deployed to the production environment.

## Step 17 – Monitoring

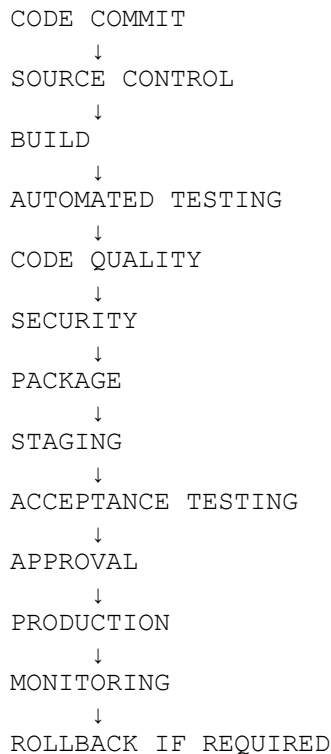
- Availability
- Errors
- Performance

## Step 18 – Rollback

If a serious production issue occurs, the system automatically or manually rolls back to the previous stable version.

## 7.3 Overall CI/CD Flow

The complete workflow can be summarized as:



## CONCLUSION

The proposed CI/CD pipeline provides a complete automated software delivery process for the **AI-Based Personal Finance and Budget Planning System (Banking & Finance)**.

The seven stages of the assignment were addressed by analyzing the project requirements, designing the CI/CD workflow, selecting appropriate tools, defining automated testing strategies, designing development/testing/staging/production environments, incorporating security and quality mechanisms, and preparing a complete CI/CD pipeline diagram.

The pipeline ensures that every code modification is automatically built, tested, analyzed, secured, packaged, and deployed through controlled environments. Quality gates prevent defective or insecure code from reaching production.

Because the project handles financial information, the inclusion of automated testing, security scanning, monitoring, notifications, backups, and rollback mechanisms is particularly important.

The final proposed process is:

**Code Commit → Build → Test → Code Quality → Security Scan → Package → Staging → Acceptance Testing → Approval → Production → Monitoring → Rollback if Required**

Thus, CI/CD improves the **quality, security, reliability, maintainability, and deployment efficiency** of the AI-Based Personal Finance and Budget Planning System.



## Rubrics for Calculation

| Evaluation Criteria                                      | Excellent (4 Marks)                                                                                              | Good (3 Marks)                                             | Satisfactory (2 Marks)                                           | Needs Improvement (1 Mark)                                         |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Requirement &amp; Pipeline Analysis(15%)</b>          | Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements                            | Identifies most relevant requirements                      | Identifies basic requirements with some gaps                     | Requirements are unclear or incomplete                             |
| <b>Pipeline Architecture &amp; Workflow(25 %)</b>        | Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages | Pipeline covers most major stages with minor gaps          | Basic pipeline is designed but several stages are missing        | Pipeline is incomplete or poorly structured                        |
| <b>Tools &amp; Technology Selection(20%)</b>             | Selects appropriate tools for each stage and provides clear justification                                        | Appropriate tools selected with reasonable justification   | Tools are identified but justification is limited                | Tools are inappropriate or not clearly identified                  |
| <b>Automated Testing &amp; Quality Integration(15 %)</b> | Effectively integrates automated testing and code-quality/security checks into the pipeline                      | Includes major testing and quality checks                  | Includes basic testing with limited integration                  | Testing/quality checks are missing or inappropriate                |
| <b>Deployment, Security &amp; Rollback Strategy(15%)</b> | Clearly designs deployment environments, security practices, monitoring, and rollback strategy                   | Covers deployment and most security/rollback aspects       | Basic deployment strategy with limited security/rollback details | Deployment strategy is incomplete or lacks security considerations |
| <b>Pipeline Diagram &amp; Documentation (10%)</b>        | Clear, accurate, professional diagram with excellent explanation and documentation                               | Well-presented diagram and documentation with minor issues | Adequate diagram/documentation but lacks clarity                 | Poorly presented, incomplete, or difficult to understand           |

| Criteria                                                 | Mark |
|----------------------------------------------------------|------|
| <b>Requirement &amp; Pipeline Analysis(15%)</b>          | /5   |
| <b>Pipeline Architecture &amp; Workflow(25%)</b>         | /4   |
| <b>Tools &amp; Technology Selection(20%)</b>             | /4   |
| <b>Automated Testing &amp; Quality Integration(15%)</b>  | /4   |
| <b>Deployment, Security &amp; Rollback Strategy(15%)</b> | /4   |
| <b>Pipeline Diagram &amp; Documentation(10%)</b>         | /4   |
| <b>TOTAL</b>                                             | /25  |